PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Software Requirements Specification

Chat Application (Socket Programming)

Version 1.0 (Draft)

Prepared by Team 5

PES University, Bangalore

04 October 2026

PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Table of Contents

1. Introduction
- **1.1 Purpose**
- **1.2 Intended Audience**
- **1.3 Product Scope**
- **1.4 References**

2. Overall Description
- **2.1 Product Perspective**
- **2.2 Product Functions**
- **2.3 User Classes and Characteristics**
- **2.4 Operating Environment**
- **2.5 Design and Implementation Constraints**
- **2.6 Assumptions and Dependencies**

3. External Interface Requirements
- **3.1 User Interfaces**
- **3.2 Software Interfaces**
- **3.3 Communications Interfaces**
- **3.4 Hardware Interfaces**

4. Analysis Models

5. System Features
- **5.1 User Registration and Authentication**
- **5.2 One-to-One Real-Time Messaging**
- **5.3 Group Chat**
- **5.4 Online Presence, Typing Indicator and Connection Handling**
- **5.5 Chat History and Persistence**
- **5.6 Delivery Status and Unread Notifications**
- **5.7 Profile Management and User Search**

6. Other Nonfunctional Requirements
- **6.1 Performance Requirements**
- **6.2 Safety Requirements**
- **6.3 Security Requirements**
- **6.4 Software Quality Attributes**
- **6.5 Business Rules**

7. Other Requirements

Appendix A: Glossary

Appendix B: Field Layouts

Appendix C: Requirement Traceability Matrix

PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Revision History

Name	Date	Reason For Changes	Version
Team 5	04-Oct-2026	Initial draft of the SRS for the Chat Application	1.0

Team Profile and Product Ownership

Team 5 (4 members) – Chat Application (Socket Programming). Every member takes part in all project activities (SRS, design, coding, testing); the table shows who owns which functional features and which quality property.

Name	SRN	Functional features owned	Qualitative property owned
Gagan Eswar	PES1UG24CS165	Feature 5.1: User Registration and Authentication (REQ-1 to REQ-7)	Security: password hashing, JWT, input validation (SEC-1 to SEC-8)
Chetha Samrutha	PES1UG24CS131	Features 5.2 and 5.5: One-to-One Messaging; Chat History and Persistence (REQ-8 to REQ-13, REQ-23, REQ-24)	Performance: message latency under 500 ms, fast history loading (PERF-1 to PERF-4)
Harsh Kalekar	PES1UG24CS183	Features 5.3 and 5.7: Group Chat; Profile Management and User Search (REQ-14 to REQ-18, REQ-29 to REQ-31)	Usability: easy-to-use, responsive user interface (Section 3.1)
Dhanush K S	PES1UG24CS148	Features 5.4 and 5.6: Presence, Typing and Connection Handling; Delivery Status and Unread Notifications (REQ-19 to REQ-22, REQ-25 to REQ-28)	Reliability: reconnection, heartbeat, no message loss (SAF-1, SAF-3)

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements of the **Real-Time Chat Application (Socket Programming)**, release 1.0, developed by Team 5 as a Mini-Project. The product lets registered users exchange text messages instantly, in one-to-one and group conversations, over persistent WebSocket connections instead of repeated HTTP polling.

This SRS covers the complete system: the browser-based client, the application server (REST API and WebSocket server) and the database. It is the baseline agreement between the team members on what will be built, and it will be kept up to date as the project progresses.

1.2 Intended Audience

This document is intended for the following readers:

Reader	What they will use this SRS for
Developers (Team 5)	Implement the features, interfaces and constraints described in Sections 3, 5 and 6.
Project guide / evaluators	Review the team's understanding of the problem, users, requirements and proposed system.
Testers	Derive test cases from the numbered requirements (REQ-n) and the traceability matrix in Appendix C.
Documentation writers / end users	Understand what the system does and the rules it follows (Sections 2, 5 and 6.5).

The rest of this document is organised as follows: Section 2 gives the overall description of the product; Section 3 specifies external interfaces; Section 4 presents analysis models (use case and entity-relationship diagrams); Section 5 lists the system features with their detailed functional requirements; Section 6 specifies non-functional requirements and business rules; Section 7 covers other requirements. Appendix A is the glossary, Appendix B gives field layouts and report requirements, and Appendix C is the requirement traceability matrix.

1.3 Product Scope

The Real-Time Chat Application is a web-based messaging system in which users register, log in, find other users and chat with them instantly. Messages are pushed to recipients over WebSocket connections (RFC 6455), so they appear without refreshing the page. The system also supports group chats, online/typing indicators, delivery/read status and persistent chat history.

Objectives and goals:

- Demonstrate socket programming concepts (persistent, full-duplex client-server connections) through a working, usable application.
- Deliver messages with low latency (typically under 500 ms on a local network).
- Keep conversations secure, private and persistent across sessions.
- Provide a clean, easy-to-use interface that works on desktop and mobile browsers.

Out of scope for release 1.0: voice/video calls, file and media sharing, end-to-end encryption, native mobile apps, message editing, and integration with external messaging services. These may be considered for future releases.

1.4 References

Ref	Title / Source	Details
[1]	IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications	IEEE, 1998
[2]	ISO/IEC/IEEE 29148:2018, Systems and software engineering – Life cycle processes – Requirements engineering	ISO/IEC/IEEE, 2018
[3]	RFC 6455 – The WebSocket Protocol	I. Fette, A. Melnikov, IETF, December 2011. https://datatracker.ietf.org/doc/html/rfc6455
[4]	RFC 7519 – JSON Web Token (JWT)	M. Jones et al., IETF, May 2015. https://datatracker.ietf.org/doc/html/rfc7519
[5]	Node.js and the 'ws' WebSocket library documentation	https://nodejs.org/docs and https://github.com/websockets/ws
[6]	OWASP Top 10 Web Application Security Risks	https://owasp.org/www-project-top-ten/
[7]	Mini-Project Guidelines and Problem Statement List (Team 5: Chat Application using WebSockets)	Course documents issued to students, PES University
[8]	SRS Template provided for the Mini-Project	Course document, PES University

2. Overall Description

2.1 Product Perspective

The Chat Application is a new, self-contained product; it does not replace or extend an existing system. It follows the familiar client-server model used by popular messaging tools, simplified for a Mini-Project. Browser clients connect to a single Node.js application server. The server exposes (a) a REST API (HTTP) for registration, login and history retrieval and (b) a WebSocket server for real-time events. Data is stored in a MongoDB database.

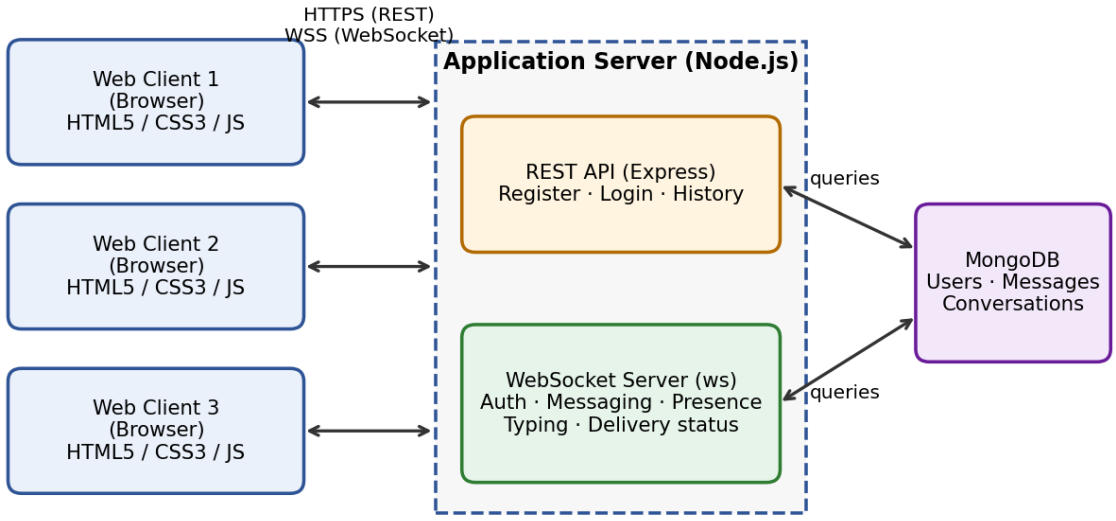


Figure 1: High-level system architecture

The major components and their responsibilities are summarised below.

Component	Responsibility	Technology
Web Client	Renders the UI, opens the WebSocket connection, sends/receives events, displays messages and status.	HTML5, CSS3, JavaScript (ES6)
REST API	User registration, login (token issue), user search, chat history retrieval, profile updates.	Node.js, Express
WebSocket Server	Authenticates connections, routes messages, broadcasts presence/typing/delivery events, heartbeat.	Node.js, ws library (WebSockets)
Database	Persistent storage of users, conversations, memberships and messages.	MongoDB (Mongoose ODM)

2.2 Product Functions

The product shall provide the following major functions (detailed in Section 5):

- **User registration and authentication** – sign up, log in/out, secure sessions.
- **One-to-one messaging** – real-time private text chat between two users.
- **Group chat** – create groups, add/remove members, broadcast messages to all members.
- **Presence and typing indicators** – online/offline status and 'user is typing…' notifications.
- **Chat history and persistence** – store all messages and load them on demand.
- **Delivery status and unread notifications** – sent / delivered / read ticks and unread counters.
- **Profile management and user search** – edit profile, change password, find users to chat with.

2.3 User Classes and Characteristics

The most important user class is the Registered User; the other classes use only a subset of the functions.

User class	Description and characteristics	Technical expertise	Frequency of use	Priority
Registered User (primary)	Any person with an account who sends and receives messages, joins groups, edits own profile.	Basic – can use a web browser	High (daily)	Most important
Group Admin	A Registered User who created a group; may add/remove members and rename the group.	Basic	Medium	Important
Visitor	A person without an account or not yet logged in; may only register or log in.	Basic	Once / occasional	Less important
System Administrator	Team member who deploys, configures and monitors the server and database (no separate UI in v1.0).	High – Node.js, MongoDB, Linux	Low	Less important

2.4 Operating Environment

- **Server:** Node.js 20 LTS (or later) on Windows 10/11, Ubuntu 22.04+ or macOS; minimum 2 GB RAM, 1 GB free disk; MongoDB 6.0 or later.
- **Client:** latest two versions of Google Chrome, Mozilla Firefox, Microsoft Edge or Safari (all support the WebSocket API); screen width from 360 px (mobile) upwards; JavaScript enabled.
- **Network:** TCP/IP connectivity between clients and server (default port 3000; HTTP/HTTPS and WS/WSS). The system must coexist with other applications on the host and use no privileged ports.

2.5 Design and Implementation Constraints

- Real-time communication must use the **WebSocket protocol** (not long-polling), as required by the problem statement.

- The back end uses JavaScript on **Node.js with WebSockets** (one of the languages/stacks permitted by the problem statement). The Python Flask-SocketIO alternative is not used.
- Only free and open-source libraries and tools may be used (MIT/Apache/BSD licences).
- Passwords must be stored only as salted hashes; secrets (JWT key, DB credentials) must be kept in environment variables, not in source code.
- Source code must be maintained in a shared Git repository and follow a common style (ESLint rules, meaningful names, comments for non-trivial logic).
- The project must be completed within the Mini-Project schedule; therefore the system is limited to a single-server deployment (no clustering) in release 1.0.
- Every team member must contribute to the SRS, design, implementation and testing, as required by the project guidelines.

2.6 Assumptions and Dependencies

Assumptions

- Users have a modern web browser and a stable internet/LAN connection.
- The server host has enough CPU, memory and bandwidth for up to about 100 concurrent users (demo scale).
- Users are identified only by username/email and password; no third-party identity provider is used.
- Messages are plain UTF-8 text (including emoji); no attachments in v1.0.
- Technology choices (Node.js, ws, Express, MongoDB) are final unless the team agrees to change them and updates this SRS.

Dependencies

- Node.js runtime and npm packages: express, ws, mongoose, bcrypt, jsonwebtoken, dotenv, validator/sanitize libraries.
- MongoDB server (local instance or MongoDB Atlas free tier) must be available and reachable.
- TLS certificates (self-signed for demo or Let's Encrypt) are needed to serve wss:// in a hosted deployment.
- Browser support for the WebSocket API (RFC 6455) is required on the client side.

3. External Interface Requirements

3.1 User Interfaces

The user interface is a single-page web application. The main screens are listed below.

Screen	Main elements
Register / Login	Username/email, password fields, 'Register' and 'Login' buttons, inline validation messages, link to switch between the two forms.
Chat Home	Left sidebar: search box, list of conversations (with unread badges) and online users. Centre: message pane with sender, text, time and status ticks. Bottom: message input box and Send button. Top bar: conversation name, online/typing status, logout.
Create / Manage Group	Dialog to enter group name and select members; group info panel for admin actions (add/remove member, leave group).
Profile / Settings	Display name, status message, change-password form.

Screen	Main elements
Notifications	Non-blocking toast messages for errors (red), success (green) and connection status (reconnecting…).

UI standards:

- Responsive layout (desktop and mobile widths); consistent colour scheme and fonts on all screens.
- Standard controls on every screen: header with app name and Logout, visible connection-status indicator, Help/About link.
- Keyboard shortcuts: Enter = send message, Shift+Enter = new line, Esc = close dialog.
- Error messages are shown in plain language next to the relevant field or as a toast; technical details are never shown to users.
- Sufficient colour contrast and labels on all inputs (basic accessibility).

Detailed screen designs (wireframes) will be documented separately in the design document.

3.2 Software Interfaces

The product interacts with the following software components.

Component	Version	Purpose
Node.js	20 LTS or later	Server runtime environment
Express	4.x	REST API (registration, login, history, search)
ws	8.x	WebSocket server implementation
MongoDB / Mongoose	MongoDB 6.x+ / Mongoose 7.x+	Persistent storage of users, conversations and messages
bcrypt	5.x	Password hashing
jsonwebtoken	9.x	Issue and verify signed session tokens (JWT)
Web browser	Chrome, Firefox, Edge, Safari (latest 2 versions)	Client runtime and WebSocket API

All real-time messages exchanged between client and server are JSON objects of the form {"type": <event>, "payload": {…}}. The main events are:

Event type	Direction	Key payload fields	Purpose
auth	Client → Server	token	Authenticate the WebSocket connection
message:send	Client → Server	conversationId, content, clientMsgId	Send a new message
message:new	Server → Client	messageId, conversationId, senderId, content, sentAt	Deliver a message to a recipient
message:status	Both	messageId, status (SENT / DELIVERED / READ)	Update delivery/read status
typing	Both	conversationId, userId, isTyping	Typing indicator
presence	Server → Client	userId, status (ONLINE / OFFLINE), lastSeen	Online/offline updates
group:update	Server → Client	conversationId, change, userId	Member added/removed, group renamed
error	Server → Client	code, message	Report invalid input or authorisation failure

3.3 Communications Interfaces

- **Protocols:** HTTP/1.1 (HTTPS in deployment) for REST calls and WebSocket (RFC 6455) over ws:// during development and wss:// (TLS 1.2 or higher) in deployment.
- **Message format:** UTF-8 encoded JSON; maximum message content length 2,000 characters; maximum frame payload 4 KB.
- **Handshake and authentication:** the client opens the WebSocket connection with a valid JWT; the server verifies the token and the Origin header before accepting the connection.
- **Connection health:** the server sends a ping every 30 seconds; a client that does not reply with a pong within 10 seconds is treated as disconnected.
- **Reconnection:** on connection loss the client retries automatically with exponential back-off (1 s, 2 s, 4 s … up to 30 s) and re-synchronises missed messages from the server.
- **Ordering and synchronisation:** the server assigns the authoritative timestamp and message ID; messages in a conversation are displayed in server-timestamp order.

3.4 Hardware Interfaces

The product has no dedicated hardware interfaces. It uses only the standard network interface of the server and client devices through the operating system's TCP/IP stack.

4. Analysis Models

Figure 2 shows the main actors and use cases. The use case for sending a private message, the core feature, is described in detail below.

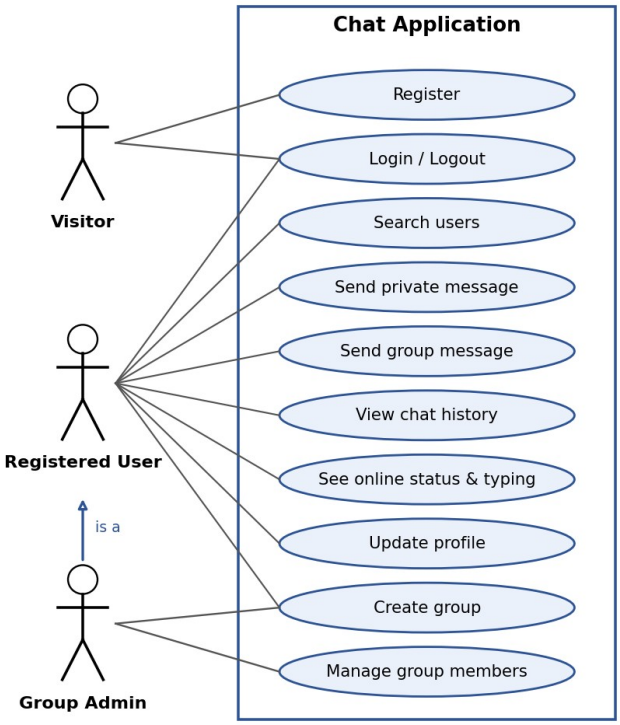


Figure 2: Use case diagram

Item	Description
Use case	UC-4: Send private message
Actor	Registered User (sender); Registered User (recipient) as secondary actor
Pre-conditions	Sender is logged in with an open authenticated WebSocket connection; a private conversation with the recipient exists or can be created.
Main flow	1. Sender selects a user and types a message. 2. Client sends a message:send event. 3. Server validates the token, content and membership. 4. Server stores the message in the database. 5. Server pushes message:new to the recipient if online. 6. Server returns SENT/DELIVERED status to the sender. 7. Recipient's client displays the message.
Alternate flow	Recipient offline: message is stored and delivered when the recipient next connects (status stays SENT until then).
Exception flow	Invalid/empty/too-long content or expired token: server returns an error event and the message is not stored.
Post-condition	Message is persisted and visible in the conversation history of both users.

The logical data model is shown in Figure 3. Field-level details are given in Appendix B.

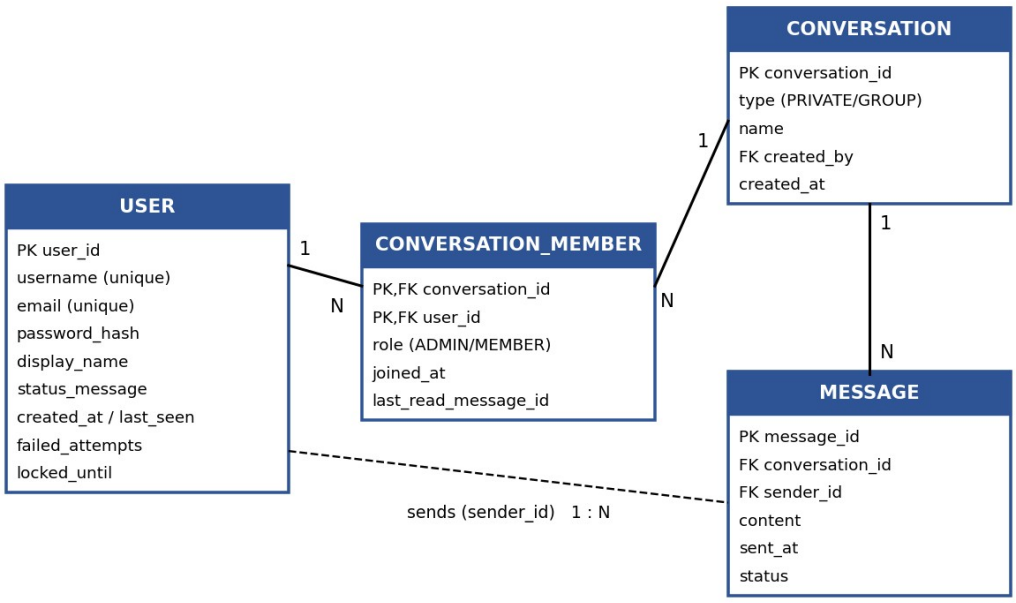


Figure 3: Entity-relationship diagram

5. System Features

The functional requirements are organised by system feature. Each requirement has a unique identifier (REQ-n) that is used in the traceability matrix (Appendix C). Seven features are specified; priorities are High, Medium or Low, with benefit, penalty, cost and risk rated from 1 (low) to 9 (high).

5.1 User Registration and Authentication

5.1.1 Description and Priority

Allows visitors to create an account and registered users to log in and out securely. Every other feature depends on it. **Priority: High** (Benefit 9, Penalty 9, Cost 4, Risk 4).

5.1.2 Stimulus/Response Sequences

Stimulus (user action)	System response
Visitor submits the registration form with valid data	System creates the account, stores a hashed password and shows a success message with a link to log in.
Visitor submits a duplicate username/email or weak password	System rejects the request and shows a specific validation error.
User submits correct credentials	System issues a JWT, opens the WebSocket connection and displays Chat Home.
User submits wrong credentials	System shows 'Invalid credentials' and counts the failed attempt.
User clicks Logout	System closes the WebSocket connection, clears the token and shows the Login screen.

5.1.3 Functional Requirements

REQ-1: The system shall allow a visitor to register with a unique username (3–20 characters), a valid email address and a password.

REQ-2: The system shall reject registration when the username or email already exists, the email format is invalid, or the password is shorter than 8 characters or lacks a letter and a digit, and shall display the reason.

REQ-3: The system shall store passwords only as salted bcrypt hashes and never in plain text.

REQ-4: The system shall authenticate users by username or email and password and issue a signed JWT valid for 24 hours.

REQ-5: The system shall show a generic 'Invalid credentials' message on failed login and lock the account for 15 minutes after 5 consecutive failed attempts.

REQ-6: The system shall accept a WebSocket connection only if it presents a valid, unexpired token; otherwise it shall close the connection with an authentication error.

REQ-7: The system shall allow a logged-in user to log out, which closes the user's WebSocket connection and discards the client-side token.

5.2 One-to-One Real-Time Messaging

5.2.1 Description and Priority

Lets two registered users exchange text messages instantly through the WebSocket connection. This is the core feature of the product. **Priority: High** (Benefit 9, Penalty 9, Cost 5, Risk 5).

5.2.2 Stimulus/Response Sequences

Stimulus (user action)	System response
User selects another user from the list or search results	System opens (or creates) the private conversation and loads recent messages.

Stimulus (user action)	System response
User types a message and presses Enter / Send	System validates and sends the message; it appears immediately in the sender's pane.
Recipient is online	Message is pushed and displayed in the recipient's pane in real time.
Recipient is offline	Message is stored and delivered the next time the recipient connects.
User sends an empty or over-long message	System blocks sending and shows an error.

5.2.3 Functional Requirements

REQ-8: The system shall allow a logged-in user to open a private conversation with any other registered user.

REQ-9: The system shall deliver a text message to an online recipient in real time over WebSocket, without a page refresh.

REQ-10: The system shall accept messages of 1 to 2,000 characters, reject empty or whitespace-only messages, and show an error if the limit is exceeded.

REQ-11: The system shall store messages addressed to an offline user and deliver them when that user next connects.

REQ-12: The system shall display each message with the sender, timestamp and status, in chronological order, and shall visually distinguish the user's own messages.

REQ-13: The system shall sanitise message content on the server and client so that scripts or HTML cannot be executed (XSS prevention).

5.3 Group Chat

5.3.1 Description and Priority

Lets users create named groups and chat with several members at once; the creator becomes the group admin. **Priority: High** (Benefit 8, Penalty 7, Cost 6, Risk 5).

5.3.2 Stimulus/Response Sequences

Stimulus (user action)	System response
User opens 'Create group', enters a name and selects members	System creates the group, makes the user admin and notifies the members.
Member sends a message in the group	System broadcasts the message to all online members and stores it for offline members.
Admin adds or removes a member	System updates the membership and sends a group:update event to all members.
Member clicks 'Leave group'	System removes the member and stops delivering group messages to them.
A non-member tries to read or post in a group	System rejects the request with an authorisation error.

5.3.3 Functional Requirements

REQ-14: The system shall allow a registered user to create a group with a name of 3–50 characters and at least two other members; the creator becomes the group admin.

REQ-15: The system shall broadcast every group message in real time to all online members and store it for offline members.

REQ-16: The system shall allow the group admin to add and remove members and to rename the group, and allow any member to leave the group.

REQ-17: The system shall display the sender's name on every group message.

REQ-18: The system shall allow only current group members to read or send messages in that group.

5.4 Online Presence, Typing Indicator and Connection Handling

5.4.1 Description and Priority

Shows who is online and who is typing, and keeps connections healthy and recoverable. **Priority: Medium** (Benefit 6, Penalty 4, Cost 4, Risk 4).

5.4.2 Stimulus/Response Sequences

Stimulus (user action)	System response
User connects or disconnects	System broadcasts a presence event; contacts see the status change.
User starts typing in a conversation	Other participants see '<name> is typing…'.
User stops typing for 3 seconds	The typing indicator disappears.
Network connection drops	Client shows 'Reconnecting…' and retries; server marks the user offline if heartbeat fails.

5.4.3 Functional Requirements

REQ-19: The system shall show each contact as online or offline (with last-seen time) and update the status within 2 seconds of a connect or disconnect.

REQ-20: The system shall show a 'user is typing…' indicator to other participants of the conversation and clear it after 3 seconds of inactivity or when the message is sent.

REQ-21: The system shall detect broken connections using a 30-second ping/pong heartbeat and mark the user offline within 40 seconds.

REQ-22: The system shall automatically try to reconnect with exponential back-off and display the connection status to the user.

5.5 Chat History and Persistence

5.5.1 Description and Priority

Stores every message permanently and lets users review past conversations. **Priority: High** (Benefit 8, Penalty 8, Cost 4, Risk 3).

5.5.2 Stimulus/Response Sequences

Stimulus (user action)	System response
User opens a conversation	System displays the latest 50 messages.
User scrolls to the top of the conversation	System loads the next 50 older messages.
User logs in again after a break	All earlier conversations and messages are still available.

5.5.3 Functional Requirements

REQ-23: The system shall persist every message with its conversation, sender, content, timestamp and status before acknowledging it to the sender.

REQ-24: The system shall load the latest 50 messages when a conversation is opened and load older messages in pages of 50 when the user scrolls up.

5.6 Delivery Status and Unread Notifications

5.6.1 Description and Priority

Gives senders feedback on message delivery and alerts users to new messages in conversations they are not viewing. **Priority: Medium** (Benefit 6, Penalty 4, Cost 4, Risk 3).

5.6.2 Stimulus/Response Sequences

Stimulus (user action)	System response
Server accepts a message	Sender sees the 'Sent' tick.
Message reaches the recipient's client	Sender sees 'Delivered' (double tick).
Recipient opens the conversation	Sender sees 'Read'; recipient's unread badge resets to zero.
New message arrives in a conversation that is not open	Unread badge increases and a notification is shown.

5.6.3 Functional Requirements

REQ-25: The system shall show the status SENT, DELIVERED or READ for every message in a one-to-one conversation.

REQ-26: The system shall update message status in real time when the recipient receives or reads the message.

REQ-27: The system shall display an unread-message count on each conversation in the list and reset it when the conversation is opened.

REQ-28: The system shall show an in-app notification (and update the browser tab title) when a message arrives in a conversation that is not currently open.

5.7 Profile Management and User Search

5.7.1 Description and Priority

Lets users maintain their profile and find other users to chat with. **Priority: Low** (Benefit 4, Penalty 2, Cost 3, Risk 2).

5.7.2 Stimulus/Response Sequences

Stimulus (user action)	System response
User edits display name or status message and saves	System validates and stores the new values and updates them for contacts.
User submits the change-password form with the correct current password	System updates the password hash and confirms.
User types in the search box	System lists matching users (partial, case-insensitive match on username).

5.7.3 Functional Requirements

REQ-29: The system shall allow a user to view and edit their display name (up to 40 characters) and status message (up to 100 characters).

REQ-30: The system shall allow a user to change their password only after verifying the current password, and the new password shall satisfy the password rules.

REQ-31: The system shall allow a user to search registered users by partial, case-insensitive username and start a conversation from the results.

6. Other Nonfunctional Requirements

6.1 Performance Requirements

PERF-1: For at least 95% of messages, end-to-end delivery latency (send to display) shall be 500 ms or less on a local network and 1 second or less on a typical broadband connection.

PERF-2: The server shall support at least 100 simultaneous WebSocket connections on a single machine (2 GB RAM) without message loss.

PERF-3: Login and registration shall complete within 2 seconds under normal load.

PERF-4: Loading the latest 50 messages of a conversation shall take 1 second or less.

PERF-5: The initial page shall load within 3 seconds on a 4 Mbps connection.

Rationale: instant delivery is the main value of a real-time chat application; the 100-connection target matches the expected demonstration scale.

6.2 Safety Requirements

SAF-1: Messages shall be written to the database before delivery is acknowledged, so that a server crash does not lose accepted messages.

SAF-2: The system shall limit each user to 20 messages per 10 seconds (rate limiting) to protect the server and other users from flooding/spam.

SAF-3: The system shall fail gracefully: if the database or WebSocket server is unavailable, users shall see a clear message and no data shall be corrupted.

SAF-4: The team shall keep a regular backup of the database (at least before each demonstration), and a restore procedure shall be documented.

SAF-5: The application is not safety-critical; no external safety certification is required.

6.3 Security Requirements

SEC-1: All users must be authenticated (username/email + password) before sending or receiving messages; WebSocket connections must present a valid JWT.

SEC-2: Passwords shall be hashed with bcrypt (cost factor 10 or higher); password hashes and tokens shall never be written to logs.

SEC-3: In deployment, all HTTP and WebSocket traffic shall be encrypted using TLS 1.2 or higher (https:// and wss://).

SEC-4: All user input shall be validated and sanitised on the server; database access shall use the ODM/parameterised queries to prevent injection attacks.

SEC-5: Every request/event shall be authorised: a user may read or write only conversations of which they are a member.

SEC-6: The server shall check the Origin header during the WebSocket handshake to prevent cross-site WebSocket hijacking.

SEC-7: Tokens expire after 24 hours; accounts are locked for 15 minutes after 5 failed logins.

SEC-8: Personal data (email, messages) shall be used only for the operation of the chat service and shall not be shared with third parties. The design shall follow the OWASP Top 10 recommendations.

6.4 Software Quality Attributes

Attribute	Requirement
Usability	A new user shall be able to register and send a first message within 2 minutes without training; at least 90% of test users shall complete this task.
Reliability	No loss of accepted messages; the server shall run for a full demonstration session (at least 4 hours) without restart; the client shall recover automatically from short network drops.
Availability	Target availability of 99% during scheduled demonstration/testing periods.
Performance efficiency	As specified in Section 6.1.
Maintainability	Modular code (separate auth, messaging, presence and data layers), consistent style enforced by ESLint, and comments on non-trivial logic.
Testability	Unit tests for core logic (target 70% coverage of server logic) and integration tests for REST and WebSocket flows.
Portability	Server runs on Windows, Linux and macOS; client runs in all supported browsers without plug-ins.
Scalability	Stateless REST layer; WebSocket layer can later be scaled horizontally with a message broker (for example Redis pub/sub).
Interoperability	Plain JSON messages over standard WebSocket/HTTP so that other clients can be written later.
Relative preference	Reliability and usability take precedence over scalability and advanced features in release 1.0.

6.5 Business Rules

BR-1: Only registered and authenticated users may use the chat functions; visitors may only register or log in.

BR-2: Usernames and email addresses must be unique (usernames are case-insensitive).

BR-3: A user can see only the conversations they participate in.

BR-4: Only the group admin may add or remove group members or rename the group; any member may leave.

BR-5: A message may contain 1–2,000 characters of text; messages cannot be edited after sending in release 1.0.

BR-6: Accounts are temporarily locked after 5 consecutive failed login attempts.

Domain requirements

- Message timestamps are generated by the server in UTC and shown in the user's local time zone.
- Within a conversation, messages are ordered by server timestamp; each message has a unique ID so that duplicates caused by re-sends or reconnection are ignored (at-least-once delivery with de-duplication).
- Text is stored and transmitted as UTF-8 so that all languages and emoji are supported.
- WebSocket connection life-cycle (handshake, open, ping/pong, close) shall follow RFC 6455.
- Delivery states follow the usual messaging convention: SENT → DELIVERED → READ.

7. Other Requirements

- **Database requirements:** MongoDB collections users, conversations, conversationMembers and messages (see Appendix B); unique indexes on username and email; index on (conversationId, sentAt) for fast history retrieval; messages are retained until the account/conversation is deleted.
- **Internationalisation:** the user interface is in English in release 1.0; all data is stored as UTF-8 so that other languages can be added later.
- **Legal and licensing:** the system is an academic project; only open-source libraries with permissive licences are used, and no copyrighted content is included.
- **Reuse:** well-known open-source libraries (Express, ws, Mongoose, bcrypt, jsonwebtoken) are reused; the modules written by the team (auth, messaging, presence) are designed to be reusable in later projects.
- **Documentation deliverables:** this SRS, a design document, a README with installation and run instructions, a short user guide and the test report.
- **Testing and deployment:** unit and integration tests, a basic load test with 100 simulated clients, and a one-command local start-up (npm start) with an .env.example file.

Appendix A: Glossary

Term	Definition
API	Application Programming Interface
bcrypt	Password-hashing algorithm that adds a random salt and is deliberately slow to resist brute-force attacks
Conversation	A private (two users) or group (three or more users) chat thread
ER diagram	Entity-Relationship diagram
HTTP / HTTPS	Hypertext Transfer Protocol / its encrypted (TLS) version
JSON	JavaScript Object Notation, a lightweight data-interchange format
JWT	JSON Web Token, a signed token used to prove that a user is authenticated
MongoDB	Document-oriented NoSQL database
Node.js	JavaScript runtime used for the server
ODM	Object-Document Mapper (Mongoose) used to access MongoDB
Presence	Whether a user is currently online or offline
REST	Representational State Transfer, an architectural style for HTTP APIs
SRS	Software Requirements Specification
TLS	Transport Layer Security, the encryption protocol behind HTTPS and WSS
WebSocket	Protocol (RFC 6455) that provides a persistent, full-duplex communication channel over a single TCP connection
WS / WSS	WebSocket / WebSocket Secure (over TLS)
XSS	Cross-Site Scripting, an attack that injects malicious scripts into web pages

Appendix B: Field Layouts

Field layouts and attributes of the main data entities of the system (Excel-style field specification), followed by the report requirements.

Field layouts of the data entities

USER

Field	Length	Data Type	Description	Is Mandatory
user_id	24	ObjectId	Unique identifier (primary key)	Y
username	20	String	Unique login name (3–20 characters)	Y
email	100	String	Unique email address	Y
password_hash	60	String	bcrypt hash of the password	Y
display_name	40	String	Name shown to other users	N
status_message	100	String	Short profile status text	N
created_at	8	Date	Date/time of registration	Y
last_seen	8	Date	Date/time user was last online	N
failed_attempts	1	Numeric	Consecutive failed login attempts	Y
locked_until	8	Date	Account lock expiry (if locked)	N

CONVERSATION

Field	Length	Data Type	Description	Is Mandatory
conversation_id	24	ObjectId	Unique identifier (primary key)	Y
type	7	String	PRIVATE or GROUP	Y
name	50	String	Group name (empty for private chats)	N
created_by	24	ObjectId	user_id of creator / group admin	Y
created_at	8	Date	Date/time of creation	Y

CONVERSATION_MEMBER

Field	Length	Data Type	Description	Is Mandatory
conversation_id	24	ObjectId	Conversation (foreign key)	Y
user_id	24	ObjectId	Member (foreign key)	Y
role	6	String	ADMIN or MEMBER	Y
joined_at	8	Date	Date/time member joined	Y
last_read_message_id	24	ObjectId	Last message read by the member (for unread counts)	N

MESSAGE

Field	Length	Data Type	Description	Is Mandatory
message_id	24	ObjectId	Unique identifier (primary key)	Y
conversation_id	24	ObjectId	Conversation the message belongs to	Y
sender_id	24	ObjectId	user_id of the sender	Y

Field	Length	Data Type	Description	Is Mandatory
content	2000	String	Message text (UTF-8)	Y
sent_at	8	Date	Server timestamp (UTC)	Y
status	9	String	SENT, DELIVERED or READ	Y

Report Requirements: fields to be included in each report

Chat History Report	User Activity Report
Conversation Name / Participants	Username
Conversation Type (Private / Group)	Display Name
Message ID	Registration Date
Sender Username	Last Seen
Message Content	Total Messages Sent
Sent Date and Time	Number of Conversations / Groups
Delivery Status	Account Status (Active / Locked)

Appendix C: Requirement Traceability Matrix

Design, code and test references are marked TBD and will be filled in as the project progresses.

Sl. No	Req. ID	Brief Description of Requirement	Architecture Reference	Design Reference	Code File Reference	Test Case ID	System Test Case ID
1	REQ-1	The system shall allow a visitor to register with a unique username (3–20 characters), a valid email address and a password.	Auth Module (REST API + WebSocket handshake)	TBD	TBD	TBD	TBD
2	REQ-2	The system shall reject registration when the username or email already exists, the email format is invalid, or the password is shorter than 8 characters or lacks a letter and a digit, and shall display the reason.	Auth Module (REST API + WebSocket handshake)	TBD	TBD	TBD	TBD
3	REQ-3	The system shall store passwords only as salted bcrypt hashes and never in plain text.	Auth Module (REST API + WebSocket handshake)	TBD	TBD	TBD	TBD
4	REQ-4	The system shall authenticate users by username or email and password and issue a signed JWT valid for 24 hours.	Auth Module (REST API + WebSocket handshake)	TBD	TBD	TBD	TBD
5	REQ-5	The system shall show a generic 'Invalid credentials' message on failed login and lock the account for 15 minutes after 5 consecutive failed attempts.	Auth Module (REST API + WebSocket handshake)	TBD	TBD	TBD	TBD
6	REQ-6	The system shall accept a WebSocket connection only if it presents a valid, unexpired token; otherwise it shall close the connection with an authentication error.	Auth Module (REST API + WebSocket handshake)	TBD	TBD	TBD	TBD
7	REQ-7	The system shall allow a logged-in user to log out, which closes the user's WebSocket connection and discards the client-side token.	Auth Module (REST API + WebSocket handshake)	TBD	TBD	TBD	TBD
8	REQ-8	The system shall allow a logged-in user to open a private conversation with any other registered user.	Messaging Module (WebSocket server)	TBD	TBD	TBD	TBD
9	REQ-9	The system shall deliver a text message to an online recipient in real time over WebSocket, without a page refresh.	Messaging Module (WebSocket server)	TBD	TBD	TBD	TBD
10	REQ-10	The system shall accept messages of 1 to 2,000 characters, reject empty or whitespace-only messages, and show an error if the limit is exceeded.	Messaging Module (WebSocket server)	TBD	TBD	TBD	TBD
11	REQ-11	The system shall store messages addressed to an offline user and deliver them when that user next connects.	Messaging Module (WebSocket server)	TBD	TBD	TBD	TBD
12	REQ-12	The system shall display each message with the sender, timestamp and status, in chronological order, and shall visually distinguish the user's own messages.	Messaging Module (WebSocket server)	TBD	TBD	TBD	TBD
13	REQ-13	The system shall sanitise message content on the server and client so that scripts or HTML cannot be executed (XSS prevention).	Messaging Module (WebSocket server)	TBD	TBD	TBD	TBD

Sl. No	Req. ID	Brief Description of Requirement	Architecture Reference	Design Reference	Code File Reference	Test Case ID	System Test Case ID
14	REQ-14	The system shall allow a registered user to create a group with a name of 3–50 characters and at least two other members; the creator becomes the group admin.	Group Module (REST + WebSocket)	TBD	TBD	TBD	TBD
15	REQ-15	The system shall broadcast every group message in real time to all online members and store it for offline members.	Group Module (REST + WebSocket)	TBD	TBD	TBD	TBD
16	REQ-16	The system shall allow the group admin to add and remove members and to rename the group, and allow any member to leave the group.	Group Module (REST + WebSocket)	TBD	TBD	TBD	TBD
17	REQ-17	The system shall display the sender's name on every group message.	Group Module (REST + WebSocket)	TBD	TBD	TBD	TBD
18	REQ-18	The system shall allow only current group members to read or send messages in that group.	Group Module (REST + WebSocket)	TBD	TBD	TBD	TBD
19	REQ-19	The system shall show each contact as online or offline (with last-seen time) and update the status within 2 seconds of a connect or disconnect.	Presence Module (WebSocket server, heartbeat)	TBD	TBD	TBD	TBD
20	REQ-20	The system shall show a 'user is typing…' indicator to other participants of the conversation and clear it after 3 seconds of inactivity or when the message is sent.	Presence Module (WebSocket server, heartbeat)	TBD	TBD	TBD	TBD
21	REQ-21	The system shall detect broken connections using a 30-second ping/pong heartbeat and mark the user offline within 40 seconds.	Presence Module (WebSocket server, heartbeat)	TBD	TBD	TBD	TBD
22	REQ-22	The system shall automatically try to reconnect with exponential back-off and display the connection status to the user.	Presence Module (WebSocket server, heartbeat)	TBD	TBD	TBD	TBD
23	REQ-23	The system shall persist every message with its conversation, sender, content, timestamp and status before acknowledging it to the sender.	History Module (REST API + database)	TBD	TBD	TBD	TBD
24	REQ-24	The system shall load the latest 50 messages when a conversation is opened and load older messages in pages of 50 when the user scrolls up.	History Module (REST API + database)	TBD	TBD	TBD	TBD
25	REQ-25	The system shall show the status SENT, DELIVERED or READ for every message in a one-to-one conversation.	Status/Notification Module (WebSocket)	TBD	TBD	TBD	TBD
26	REQ-26	The system shall update message status in real time when the recipient receives or reads the message.	Status/Notification Module (WebSocket)	TBD	TBD	TBD	TBD
27	REQ-27	The system shall display an unread-message count on each conversation in the list and reset it when the conversation is opened.	Status/Notification Module (WebSocket)	TBD	TBD	TBD	TBD
28	REQ-28	The system shall show an in-app notification (and update the browser tab title) when a message arrives in a conversation that is not currently open.	Status/Notification Module (WebSocket)	TBD	TBD	TBD	TBD

Sl. No	Req. ID	Brief Description of Requirement	Architecture Reference	Design Reference	Code File Reference	Test Case ID	System Test Case ID
29	REQ-29	The system shall allow a user to view and edit their display name (up to 40 characters) and status message (up to 100 characters).	Profile/Search Module (REST API)	TBD	TBD	TBD	TBD
30	REQ-30	The system shall allow a user to change their password only after verifying the current password, and the new password shall satisfy the password rules.	Profile/Search Module (REST API)	TBD	TBD	TBD	TBD
31	REQ-31	The system shall allow a user to search registered users by partial, case-insensitive username and start a conversation from the results.	Profile/Search Module (REST API)	TBD	TBD	TBD	TBD